

TJSC 2026 - Analista de Sistemas

Segurança da Informação e Governança de TI

Aula 4

SSO, Keycloak, autenticação federada, sessões e tokens

Campo	Detalhe
Disciplina	Segurança da Informação e Governança de TI
Trilha	Aula 4 de 11
Recorte	Single Sign On - SSO; Keycloak; autenticação federada; sessões; tokens; distinções com OAuth2/OIDC apenas como apoio
Foco	Acertar questões FGV que misturam conceito, cenário de sistema corporativo, IdP, aplicação, API, sessão e token
Formato	Teoria objetiva, quadros comparativos, pegadinhas, 20 questões autorais, gabarito separado e comentários completos

Material original, orientado ao edital retificado do TJSC 2026 e calibrado por cadernos públicos da FGV e fontes primárias. As questões são autorais e inéditas.

1. Identificação da aula e recorte do edital

Esta é a Aula 4 da trilha nova de Segurança da Informação e Governança de TI. Ela cobre o item 1.4 Single Sign On - SSO e o item 1.5 Keycloak do edital, além dos conceitos indispensáveis para entender autenticação federada, sessões e tokens. Os itens OAuth2 RFC 6749 e OpenID Connect - OIDC aparecem aqui apenas como suporte, porque serão aprofundados na Aula 5.

A FGV já cobrou, em prova recente de tribunal, afirmações envolvendo SAML, SSO e Keycloak, inclusive a ideia de que Keycloak permite implementar SSO e suporta OpenID Connect e OAuth 2.0. Portanto, este tema não deve ser tratado como detalhe periférico: ele é expressamente edital e já apareceu em cobrança técnica aplicada.

Objetivo da aula: ao final, você deve conseguir ler um cenário com usuário, navegador, aplicação Angular, API Java/Quarkus, Keycloak, provedor de identidade, token e sessão - e identificar exatamente quem autentica, quem autoriza, quem mantém sessão e quem valida token.

2. Mapa mental textual da aula

SSO E KEYCLOAK

- SSO - Single Sign On: autenticação centralizada para múltiplas aplicações confiantes.
- IdP - Identity Provider: provedor de identidade que autentica o usuário.
- SP/RP - Service Provider/Relying Party: aplicação que confia no IdP.
- Keycloak: servidor de identidade e acesso que protege aplicações por protocolos abertos.
- Autenticação federada: confiança entre domínios/organizações/sistemas para aceitar identidade externa.
- Identity brokering: Keycloak atuando como intermediário entre aplicação e IdP externo.
- Sessão: estado de login mantido por servidor/aplicação/IdP, normalmente com cookie associado.
- Token: artefato digital apresentado para provar autenticação, autorização ou identidade.
- Access token: apresentado à API para acesso a recurso protegido.
- ID token: informa à aplicação que o usuário foi autenticado e carrega claims de identidade.
- Refresh token: usado para obter novos tokens sem exigir novo login imediato.

3. Siglas e termos essenciais

Sigla/termo	Expansão	Como lembrar em prova
SSO	Single Sign On	Login único para acessar várias aplicações confiantes, sem redigitar credenciais a cada sistema.
IdP	Identity Provider	Quem autentica o usuário e emite token/assertion.
SP	Service Provider	No SAML, aplicação que consome a autenticação feita pelo IdP.
RP	Relying Party	No OIDC, aplicação/cliente que confia no provedor de identidade.
OIDC	OpenID Connect	Camada de autenticação sobre OAuth2; traz ID Token e claims do usuário.
OAuth2	OAuth 2.0 Authorization Framework	Framework de autorização delegada; será a Aula 5.
SAML	Security Assertion Markup Language	Padrão XML de federação e SSO, muito usado em ambiente corporativo.
JWT	JSON Web Token	Formato compacto de token com header, payload e assinatura; assinado não significa criptografado.
JWK	JSON Web Key	Formato JSON para chaves públicas usadas para verificar assinatura de tokens.
JWKS	JSON Web Key Set	Conjunto de chaves públicas publicado pelo provedor para validação de tokens.
MFA	Multi-Factor Authentication	Autenticação com múltiplos fatores: algo que sabe, possui ou é.

Sigla/termo	Expansão	Como lembrar em prova
LDAP	Lightweight Directory Access Protocol	Protocolo para acessar diretórios, como bases corporativas de usuários.
AD	Active Directory	Diretório da Microsoft, frequentemente usado como fonte corporativa de usuários.
TLS	Transport Layer Security	Protege o tráfego em trânsito; indispensável em fluxos de login e tokens.
PKCE	Proof Key for Code Exchange	Mitigação para clientes públicos; será detalhada na Aula 5.
API	Application Programming Interface	Interface acessada por sistemas; em prova, costuma validar access token.

4. Antes do SSO: identificação, autenticação, autorização, sessão e token

A FGV costuma explorar confusão entre termos próximos. Em sistemas reais, especialmente com Keycloak, o erro de prova nasce quando o candidato chama tudo de login, tudo de token ou tudo de autorização.

Conceito	Pergunta que responde	Exemplo no ambiente corporativo
Identificação	Quem você diz ser?	Informar matrícula, CPF, e-mail ou username.
Autenticação	Você prova ser quem disse ser?	Senha, MFA, certificado, biometria ou login federado validado pelo IdP.
Autorização	O que você pode acessar?	Perfil de analista, role de administrador, permissão para aprovar uma solicitação.
Sessão	O sistema lembra que você está logado?	Cookie de sessão do navegador ligado ao login no Keycloak ou na aplicação.
Token	Que artefato comprova algo?	Access token enviado como Bearer para API; ID token consumido pela aplicação.

Pegadinha central: autenticação e autorização não são sinônimos. O usuário pode estar autenticado e ainda assim não estar autorizado a acessar determinado recurso. Em uma API de pagamentos, por exemplo, estar logado não significa poder aprovar pagamentos.

Token também não é sinônimo de sessão. Uma sessão normalmente envolve estado mantido por um servidor ou provedor de identidade. Um token é um artefato apresentado por um cliente, que pode ser validado pela API. Em arquiteturas modernas, os dois convivem: o navegador tem sessão com o IdP e a API valida tokens.

5. Single Sign On - SSO

SSO - Single Sign On é o modelo em que o usuário se autentica uma vez perante uma autoridade central ou provedor de identidade e passa a acessar múltiplas aplicações que confiam nesse provedor. Em vez de cada sistema manter sua própria tela de login, senha e regra de autenticação, as aplicações delegam a autenticação a um ponto central.

Em um órgão público, isso **evita a proliferação de senhas por sistema e simplifica governança de acesso**. Em ambiente bancário, a lógica é familiar: aplicações internas diferentes podem confiar em uma infraestrutura corporativa de identidade, com políticas de senha, MFA e bloqueio centralizadas.

Aspecto	Vantagem	Risco ou cuidado
Usabilidade	Usuário evita autenticação repetida em cada aplicação.	SSO não significa sessão infinita; políticas de expiração continuam necessárias.
Segurança	MFA, política de senha e bloqueio podem ser centralizados.	Comprometimento da conta central pode ampliar impacto.
Governança	Revogação e auditoria ficam mais consistentes.	Aplicações ainda precisam validar autorização local ou por roles/scopes.
Integração	Novas aplicações podem aderir ao IdP comum.	Configuração incorreta de redirect URI, audience ou roles pode expor recursos.

A banca pode tentar vender a ideia de que SSO elimina a necessidade de autenticação. Errado. SSO não elimina autenticação; ele centraliza ou federa a autenticação. Também não elimina autorização: cada

aplicação/API ainda precisa decidir o que o usuário pode fazer.

6. Autenticação federada

Autenticação federada ocorre **quando uma aplicação aceita uma identidade autenticada por outro domínio, organização ou provedor**. Há uma relação de confiança: a aplicação não coleta a senha do usuário, mas confia na assertion ou no token emitido pelo provedor de identidade.

Modelo	Ideia	Exemplo
Autenticação local	A aplicação autentica diretamente o usuário.	Sistema próprio com tabela USUARIO e senha local.
SSO interno	Várias aplicações confiam em um provedor central da própria organização.	Aplicações internas usando Keycloak corporativo.
Federação externa	A aplicação aceita identidade de outro provedor confiável.	Keycloak delegando login a outro IdP OIDC/SAML.
Social login	Uso de provedor externo público.	Login com Google, em cenário não necessariamente adequado para sistema público sensível.

Na federação, a senha do usuário deve ficar com o provedor de identidade. A aplicação recebe um token, assertion ou conjunto de claims. Isso reduz exposição de credenciais, mas transfere parte da criticidade para a configuração de confiança, validação de assinatura, escopo, audience, expiração e mapeamento de atributos.

7. Keycloak: o que é e por que cai

Keycloak é um **servidor de gerenciamento de identidade e acesso**. Ele pode proteger aplicações e serviços usando padrões abertos como OpenID Connect e SAML 2.0. A documentação oficial destaca que as aplicações redirecionam o navegador do usuário para o servidor Keycloak, onde as credenciais são informadas; as aplicações não precisam ver a senha do usuário.

Para a FGV, o ponto mais provável não é cobrar comando de instalação. O ponto provável é reconhecer o papel do Keycloak em uma arquitetura: ele é IdP, broker de identidade, emissor/validador de tokens por configuração, gestor de usuários/roles/grupos/realms e ponto de SSO.

Termo Keycloak	Definição de prova	Cuidado
Realm	Domínio isolado de segurança, usuários, clientes, roles e configurações.	Não confundir com aplicação específica; um realm pode ter vários clients.
Client	Aplicação ou serviço registrado no Keycloak.	Pode ser uma aplicação web, SPA, backend ou API; não é necessariamente usuário humano.
User	Entidade capaz de autenticar no sistema.	Usuário tem atributos, grupos e roles.
Group	Agrupamento de usuários para facilitar administração.	Grupo pode receber roles, mas não é o mesmo que role.
Role	Papel/permissão atribuível a usuários, grupos ou clientes.	Role não autentica ninguém; ela apoia autorização.
Identity Provider	Provedor externo ao qual o Keycloak pode delegar autenticação.	Keycloak pode atuar como broker, não apenas como base local.
User federation	Integração com base externa de usuários, como LDAP/AD.	Diferente de identity brokering: aqui o Keycloak consulta/sincroniza usuários.
Token mapper	Mapeia atributos/roles para tokens/assertions.	Configuração errada pode omitir ou expor claims indevidas.

8. Identity brokering x user federation

Este é um ponto com cara de alternativa FGV: os dois termos parecem similares, mas não são equivalentes.

Conceito	O que faz	Exemplo
User federation	Conecta o Keycloak a uma fonte externa de usuários para consulta/sincronização/autenticação, como LDAP ou Active Directory.	Keycloak consulta diretório corporativo para autenticar servidor/empregado.

Conceito	O que faz	Exemplo
Identity brokering	Keycloak atua como intermediário entre a aplicação e outro provedor de identidade.	Aplicação confia no Keycloak, e o Keycloak delega login a outro IdP OIDC/SAML.
Federação de identidade	Modelo amplo de confiança entre provedores e serviços.	Órgão A confia na identidade emitida pelo órgão B, conforme acordo e protocolo.

Resumo de prova: user federation é fonte externa de usuários; identity brokering é intermediação de autenticação com outro IdP. Ambos podem existir no Keycloak, mas a finalidade operacional é diferente.

9. Sessões em SSO e Keycloak

Sessão é o estado que permite ao sistema reconhecer que o usuário já passou por autenticação. Em SSO, é comum existir uma sessão no provedor de identidade e, paralelamente, sessões ou estados nas aplicações clientes.

Exemplo prático: você acessa uma aplicação Angular. Ela redireciona para o Keycloak. O Keycloak autentica o usuário e cria uma sessão SSO. Ao voltar para a aplicação, a aplicação recebe tokens e pode manter seu próprio estado local. Se o usuário abre outra aplicação do mesmo realm, pode não precisar redigitar senha, porque ainda há sessão válida no Keycloak.

Tipo	Função	Pegadinha
Sessão SSO	Estado de login no provedor de identidade, usado para permitir login único em clients confiantes.	Expirar a sessão SSO pode exigir nova autenticação em vários clients.
Sessão da aplicação	Estado mantido pela aplicação cliente ou backend.	Logout local pode não encerrar sessão global no IdP.
Client session	Estado ligado à interação de um client específico com o usuário.	Diferente da sessão SSO global do usuário.
Offline session	Sessão/tokens para acesso prolongado sem presença ativa do usuário, quando configurado.	Deve ser usada com extremo cuidado, pois aumenta janela de risco.

Cookies de sessão devem usar atributos de segurança compatíveis com o contexto: Secure para HTTPS, HttpOnly quando o JavaScript não precisa acessar o cookie e SameSite conforme o fluxo de redirecionamento e proteção contra CSRF. A banca pode não cobrar atributo de cookie nesta aula, mas isso ajuda a resolver cenários de sessão hijacking e fixação de sessão.

10. Tokens: access token, ID token e refresh token

Token é um artefato emitido por uma autoridade e apresentado por uma aplicação/cliente para provar uma condição: autorização, autenticação ou possibilidade de renovação. Em arquiteturas com OpenID Connect, aparecem três termos que a FGV pode misturar.

Token	Uso principal	Quem normalmente consome	Erro clássico
Access token	Autorizar acesso a recurso protegido, como uma API.	Resource server/API.	Usá-lo como prova primária de identidade do usuário na aplicação cliente.
ID token	Informar autenticação do usuário e claims de identidade.	Client/Relying Party.	Enviar ID token para API como se fosse access token.
Refresh token	Obter novos tokens sem novo login imediato.	Client autorizado.	Mandá-lo para APIs ou armazená-lo sem proteção adequada.

Um JSON Web Token - JWT - assinado geralmente possui header, payload e assinatura. O payload costuma ser apenas codificado em Base64URL, não criptografado. Portanto, não se deve colocar segredo sensível em claim achando que a assinatura torna o conteúdo confidencial. Assinatura protege integridade e autenticidade do emissor; confidencialidade exige criptografia ou não exposição do dado.

11. Claims e validação de token

Claims são **afirmações transportadas em um token**: emissor, assunto, público, expiração, escopos, roles, e-mail, nome ou outros atributos. O ponto de prova não é decorar todas as claims, mas entender quais validações são críticas.

Claim	Ideia	Validação esperada
iss - issuer	Quem emitiu o token.	A API deve aceitar apenas emissor esperado, como o realm correto do Keycloak.
sub - subject	Identificador do usuário ou entidade autenticada.	Não deve ser confundido com username mutável.
aud - audience	Público/serviço para o qual o token se destina.	API não deve aceitar token emitido para outro client/resource.
exp - expiration	Momento de expiração.	Token expirado deve ser rejeitado.
iat - issued at	Momento de emissão.	Ajuda a avaliar frescor do token.
scope	Escopos concedidos.	Não basta estar autenticado: operação deve exigir escopo/role compatível.
roles	Papéis atribuídos.	Devem ser mapeados corretamente para autorização da aplicação.

Validação mínima em uma API: assinatura com chave pública correta, issuer esperado, audience compatível, expiração válida e autorização por scopes/roles. Conferir apenas se o token 'parece JWT' não é validação de segurança.

12. Fluxo mental: Angular, Quarkus, Keycloak e API

Cenário: uma aplicação Angular acessa uma API Java/Quarkus. O login é feito no Keycloak.

- 1 Usuário acessa a aplicação Angular.
- 2 A aplicação verifica que não há sessão/autorização local suficiente e redireciona o usuário para o Keycloak.
- 3 O Keycloak autentica o usuário, possivelmente com MFA e/ou provedor federado.
- 4 Após autenticação, o Keycloak retorna artefatos ao client, como código de autorização e tokens, conforme o fluxo configurado.
- 5 A aplicação passa a chamar a API com access token no cabeçalho Authorization: Bearer.
- 6 A API Quarkus valida o token: assinatura, emissor, audience, expiração e permissões.
- 7 A API decide autorização: por exemplo, role ANALISTA pode consultar; role APROVADOR pode aprovar.

Observe que o Keycloak autentica, mas a API não deve confiar cegamente no front-end. A autorização efetiva precisa existir no backend, porque o front-end pode ser manipulado pelo usuário. A FGV adora alternativas que colocam segurança apenas na camada visual.

13. Logout, revogação e expiração

Logout é outro tema de pegadinha. **Encerrar a tela do navegador não é necessariamente encerrar sessão no IdP.** Remover estado local da aplicação **não significa revogar refresh token.** **Expirar access token não invalida automaticamente uma sessão SSO ainda válida.**

Evento	Efeito provável	Cuidado
Logout local da aplicação	Remove estado da aplicação cliente.	Pode não encerrar a sessão no Keycloak.
Logout no IdP	Encerra sessão central no provedor de identidade.	Clients podem precisar receber sinalização front-channel/back-channel.
Expiração de access token	API rejeita chamadas com token vencido.	Refresh token ainda pode obter novo access token.
Revogação de refresh token	Cliente perde capacidade de renovar tokens.	Deve ser protegida e auditada em cenários sensíveis.
Not-before policy	Tokens emitidos antes de certo instante podem ser rejeitados.	Útil para revogação ampla por usuário, client ou realm.

14. Segurança prática: onde a FGV pode apertar

- SSO reduz fadiga de senha, mas cria concentração de risco. Por isso, MFA, proteção de sessão, TLS e monitoramento são relevantes.
- Token assinado não é token criptografado. O conteúdo pode ser lido se não houver criptografia; a assinatura verifica integridade e emissor.
- Refresh token tem maior criticidade que access token curto. Se vazado, pode renovar acesso.
- A API deve validar token no backend. Não basta o Angular esconder botão ou rota.
- Audience errada é falha crítica: token emitido para um client não deve abrir API indevida.
- A vida útil de tokens e sessões deve equilibrar usabilidade e risco; sistemas sensíveis tendem a usar access tokens curtos.
- Roles no token ajudam, mas a autorização deve refletir regra de negócio. Role genérica demais costuma virar privilégio excessivo.
- Keycloak mal configurado não resolve governança: redirect URIs amplas, clients públicos indevidos e mappers excessivos podem causar exposição.

15. Clients públicos, clients confidenciais e PKCE

Em Keycloak, **client é a aplicação ou serviço registrado**. A FGV pode não cobrar tela administrativa, mas pode cobrar o efeito de uma configuração errada. Um client público, como uma Single Page Application - SPA em Angular executada no navegador, não consegue guardar segredo de forma confiável. Um client confidencial, como um backend Java/Quarkus, pode guardar segredo no servidor. Essa distinção influencia o fluxo de autenticação, a proteção do código de autorização e o uso de Proof Key for Code Exchange - PKCE.

Tipo de client	Característica	Cuidado de prova
Público	Não consegue guardar client secret com segurança, pois roda em ambiente controlado pelo usuário.	SPA/mobile devem usar proteções adequadas, como Authorization Code com PKCE no contexto moderno.
Confidencial	Consegue proteger client secret no servidor.	Backend for Frontend - BFF ou aplicação server-side pode manter segredo fora do navegador.
Bearer-only/API	API não inicia login interativo; apenas valida token recebido.	API deve validar token, não redirecionar usuário como aplicação web comum.

Pegadinha: armazenar client secret em Angular não torna a SPA confidencial. Qualquer segredo embarcado em código entregue ao navegador pode ser extraído. Em prova, quando o enunciado fala em SPA, navegador ou JavaScript no cliente, pense em client público.

16. Redirect URI, origem e risco de desvio

Redirect URI é o **endereço para o qual o provedor de identidade redireciona o usuário após a etapa de autenticação/autorização**. Configuração ampla demais pode permitir que artefatos do fluxo sejam enviados para origem indevida. Por isso, aplicações devem registrar URIs específicas e previsíveis.

Configuração	Avaliação
https://sistema.tjsc.jus.br/callback	Específica e controlada; tende a ser adequada.
https://*.qualquer-dominio.com/*	Ampla demais; aumenta risco de desvio.
http://sistema.tjsc.jus.br/callback em produção	Inadequada para tráfego sensível; deve usar TLS/HTTPS.
localhost em desenvolvimento	Pode existir em ambiente de teste, mas não deve ser confundida com produção.

17. Armazenamento de tokens em aplicações web

Em aplicação Angular, o tema vira decisão de arquitetura. LocalStorage é simples, mas aumenta exposição em caso de Cross-Site Scripting - XSS, pois JavaScript malicioso pode ler o token. Cookies HttpOnly reduzem leitura por JavaScript, mas exigem cuidado contra Cross-Site Request Forgery - CSRF e configuração de SameSite. Em sistemas sensíveis, arquiteturas como Backend for Frontend - BFF podem reduzir exposição de tokens no navegador.

Estratégia	Ponto positivo	Risco/cuidado
localStorage	Fácil de implementar e persiste entre abas/sessões.	Exposto a XSS; não ideal para tokens sensíveis.
sessionStorage	Some ao fechar aba/sessão do navegador.	Ainda é acessível por JavaScript; também sofre com XSS.
Cookie HttpOnly + Secure	Não é lido por JavaScript e só trafega em HTTPS quando Secure.	Exige estratégia CSRF/SameSite e design correto.
BFF	Backend mantém tokens e front-end conversa com backend próprio.	Mais complexo, mas reduz exposição de tokens no navegador.

18. Interpretação FGV: como a banca monta a armadilha

Em Tecnologia da Informação, a FGV costuma usar contexto curto, mas com termos próximos. Em vez de perguntar 'o que é SSO?', ela pode apresentar um órgão público, um IdP, uma aplicação web, uma API e três afirmações. A questão exige distinguir papel e finalidade. O alvo não é decorar siglas: é saber que peça faz o quê.

- Quando a alternativa disser que SSO elimina autorização, elimine.
- Quando disser que Keycloak é protocolo, elimine: Keycloak é plataforma/servidor.
- Quando disser que aplicação cliente deve ver a senha do usuário no fluxo federado, elimine.
- Quando a API aceitar token sem validar assinatura/issuer/audience, a alternativa tende a estar errada.
- Quando ID token for usado como token de API, desconfie.
- Quando refresh token aparecer trafegando em toda requisição de API, desconfie.
- Quando JWT assinado for tratado como criptografado, desconfie.

19. Quadro comparativo final

Par de conceitos	Diferença que derruba candidato
SSO x autenticação	SSO é modelo de autenticação centralizada/federada; não é ausência de autenticação.
Keycloak x protocolo	Keycloak é produto/servidor IAM; OIDC, OAuth2 e SAML são padrões/protocolos.
IdP x aplicação	IdP autentica e emite token/assertion; aplicação consome e aplica regras de autorização.
Sessão x token	Sessão é estado de login; token é artefato apresentado/validado.
Access token x ID token	Access token protege API/recurso; ID token informa autenticação e identidade ao client.
Refresh token x senha	Refresh token renova tokens, mas não é senha; ainda assim é altamente sensível.
User federation x identity brokering	User federation integra fonte de usuários; brokering delega autenticação a outro IdP.
Assinatura x criptografia	Assinatura prova integridade/emissor; criptografia protege confidencialidade.

20. Pegadinhas FGV prováveis

- Dizer que SSO elimina a necessidade de autenticação ou autorização.
- Tratar Keycloak como se fosse o próprio protocolo OAuth2 ou OIDC.
- Afirmar que OAuth2 autentica usuário por si só; a camada de identidade é do OpenID Connect.

- Confundir SAML com JWT: SAML usa assertions XML; OIDC costuma usar tokens JSON/JWT.
- Dizer que access token é destinado primariamente à aplicação cliente para provar identidade; o mais correto é recurso/API.
- Dizer que ID token deve ser enviado à API como bearer token de autorização.
- Achar que JWT assinado é confidencial.
- Achar que logout local encerra automaticamente todas as sessões federadas.
- Confundir grupo com role ou role com usuário.
- Imaginar que front-end controla autorização de forma suficiente.

21. Lista de questões autorais - padrão FGV

Resolva as 20 questões sem consultar o gabarito. Há mistura de questão conceitual, caso prático, afirmativas, V/F, relação entre conceitos e interpretação de cenário técnico. As alternativas foram construídas com distratores próximos, sem gabarito anunciado no enunciado.

1. Em um projeto de modernização de sistemas internos, o TJSC pretende permitir que servidores acessem várias aplicações web após autenticação em um provedor central, evitando que cada sistema mantenha mecanismo próprio de login. O desenho proposto preserva controles de expiração de sessão, autenticação multifator e autorização por perfil em cada aplicação. Nessa situação, a característica principal do Single Sign On - SSO é:

- (A) dispensar autenticação para aplicações que estejam dentro da rede corporativa.
- (B) substituir autorização por autenticação, pois o usuário logado passa a ter acesso automático aos recursos.
- (C) centralizar ou federar a autenticação, permitindo acesso a múltiplas aplicações confiantes sem nova digitação de credenciais a cada uma.
- (D) criptografar automaticamente todos os dados trafegados entre aplicações, eliminando a necessidade de TLS.
- (E) transformar cada aplicação cliente em autoridade emissora de identidade para as demais.

2. Em uma arquitetura com Keycloak, uma aplicação Angular redireciona o usuário para autenticação e, após o login, passa a consumir uma API Java/Quarkus. Considerando o papel típico do Keycloak nesse cenário, assinale a opção correta.

- (A) O Keycloak é uma biblioteca de front-end responsável por armazenar as senhas no navegador do usuário.
- (B) O Keycloak substitui a necessidade de qualquer validação no backend, pois o front-end já recebeu tokens.
- (C) O Keycloak é um protocolo concorrente do OpenID Connect, utilizado quando OAuth2 não está disponível.
- (D) O Keycloak deve ser instalado dentro de cada aplicação cliente, pois não pode atuar como servidor separado de identidade.
- (E) O Keycloak pode atuar como servidor de identidade e acesso, protegendo aplicações por padrões como OpenID Connect e SAML.

3. Uma equipe configurou login federado no Keycloak. O usuário acessa uma aplicação do órgão, mas a autenticação é delegada a outro provedor de identidade confiável. O Keycloak recebe informações do provedor externo e as entrega à aplicação em formato consumível por ela. Esse arranjo descreve, com maior precisão:

- (A) criptografia simétrica entre cliente e servidor.
- (B) identity brokering ou intermediação de identidade.
- (C) cache distribuído de credenciais em múltiplas aplicações.
- (D) backup de sessão local para alta disponibilidade.
- (E) autorização discricionária sem provedor central.

4. Analise as afirmativas a seguir sobre SSO, federação e Keycloak. I. SSO elimina a necessidade de autorização, pois o usuário autenticado centralmente deve ser aceito por todos os recursos. II. Keycloak pode integrar aplicações por padrões abertos, como OpenID Connect e SAML. III. Em autenticação federada, a aplicação pode confiar em uma identidade emitida por provedor externo, desde que haja relação de confiança e validação adequada. Está correto o que se afirma em:

- (A) I, apenas.
- (B) I e II, apenas.
- (C) I e III, apenas.
- (D) II e III, apenas.
- (E) I, II e III.

5. No contexto de Keycloak, assinale a opção que melhor diferencia user federation de identity brokering.

- (A) User federation integra o Keycloak a uma fonte externa de usuários, como LDAP/Active Directory; identity brokering faz o Keycloak intermediar autenticação com outro provedor de identidade.
- (B) User federation é o ato de assinar JWTs; identity brokering é o ato de criptografar access tokens.

- (C) User federation ocorre apenas em aplicações monolíticas; identity brokering ocorre apenas em microsserviços.
- (D) User federation substitui o uso de realms; identity brokering substitui a necessidade de clients.
- (E) User federation é sinônimo de logout global; identity brokering é sinônimo de refresh token.

6. Em uma aplicação corporativa, o desenvolvedor recebe um JSON Web Token - JWT assinado e afirma que pode armazenar dados sigilosos no payload porque a assinatura impede sua leitura por terceiros. A afirmação está:

- (A) correta, pois toda assinatura digital implica criptografia do conteúdo assinado.
- (B) correta, desde que o token seja transportado por HTTP sem TLS.
- (C) incorreta, pois a assinatura protege integridade e autenticidade do emissor, mas não garante confidencialidade do payload.
- (D) incorreta apenas se o token for usado em OIDC; em OAuth2 o payload é sempre criptografado.
- (E) correta quando a claim aud estiver preenchida.

7. Relacione os conceitos aos seus usos principais. 1. Access token 2. ID token 3. Refresh token () Usado para obter novos tokens, conforme política do provedor. () Consumido pela aplicação cliente para obter informações de autenticação/identidade do usuário. () Apresentado a uma API/recurso protegido para autorizar acesso. A sequência correta é:

- (A) 1 - 2 - 3
- (B) 3 - 2 - 1
- (C) 2 - 1 - 3
- (D) 2 - 3 - 1
- (E) 3 - 1 - 2

8. Um analista verifica que uma API aceita qualquer token com estrutura aparente de JWT, desde que o campo exp ainda esteja no futuro. Ela não valida assinatura, issuer nem audience. Em uma revisão de segurança, a falha mais relevante é:

- (A) ausência de tabela local de sessões na API, pois APIs REST nunca podem validar tokens diretamente.
- (B) uso de expiração em tokens, pois tokens válidos não devem expirar em arquiteturas SSO.
- (C) uso de SSO, pois SSO impede autorização granular por API.
- (D) uso de claims, pois claims não podem transportar qualquer informação de identidade.
- (E) validação insuficiente do token, pois a API deve verificar assinatura, emissor, público, expiração e autorização aplicável.

9. A respeito de sessão e token em sistemas com SSO, assinale a opção correta.

- (A) Sessão e token são conceitos idênticos: ambos significam exclusivamente senha armazenada no navegador.
- (B) Um token válido dispensa sempre a verificação de autorização, pois o usuário já foi autenticado.
- (C) Uma sessão de aplicação local encerra, por definição, todas as sessões existentes no provedor de identidade.
- (D) Sessão é estado de login mantido por aplicação ou provedor; token é artefato apresentado para provar autorização, autenticação ou renovação, conforme o tipo.
- (E) Em SSO não há sessões, apenas arquivos de configuração compartilhados.

10. Durante a integração com Keycloak, uma equipe decide que a aplicação front-end será a única responsável por esconder botões e menus de administração. A API aceitará as requisições sem verificar roles ou escopos, pois a tela já impede usuários comuns de clicar nas ações. A decisão é inadequada porque:

- (A) a autorização deve ser validada também no backend/API, já que o front-end pode ser manipulado ou contornado.
- (B) aplicações Angular não podem usar Keycloak em nenhuma hipótese.
- (C) roles só podem ser aplicadas em bancos de dados relacionais, não em APIs.
- (D) OAuth2 exige que toda autorização seja feita exclusivamente por cookies de sessão.
- (E) SSO impede o uso de qualquer controle de acesso em aplicação.

11. Considere as afirmações sobre realms, clients e roles no Keycloak. I. Realm representa um domínio isolado de segurança, com usuários, clients, roles e configurações próprias. II. Client corresponde a uma aplicação ou serviço registrado para integração com o Keycloak. III. Role autentica o usuário, substituindo credenciais como senha ou MFA. Está correto o que se afirma em:

- (A) I, apenas.
- (B) II, apenas.
- (C) I e II, apenas.
- (D) II e III, apenas.
- (E) I, II e III.

12. Em uma arquitetura de SSO, o logout local de uma aplicação remove o estado dessa aplicação, mas o usuário consegue entrar em outra aplicação do mesmo domínio sem digitar senha novamente. A melhor explicação para o ocorrido é:

- (A) a sessão central no provedor de identidade provavelmente permaneceu válida, de modo que o logout local não encerrou o SSO global.
- (B) o access token foi transformado automaticamente em senha permanente.
- (C) o Keycloak não suporta logout, apenas autenticação inicial.
- (D) a API manteve a autorização porque tokens JWT nunca expiram.
- (E) o usuário foi autenticado pela API de banco de dados, não por provedor de identidade.

13. A equipe de segurança definiu que o access token emitido para a aplicação de consulta processual não deve ser aceito pela API de administração de usuários. A validação mais diretamente relacionada a esse requisito é:

- (A) iat - issued at, pois define o nome do usuário.
- (B) sub - subject, pois identifica o algoritmo de assinatura.
- (C) exp - expiration, pois indica o servidor LDAP usado na autenticação.
- (D) aud - audience, pois indica o público/recurso ao qual o token se destina.
- (E) nonce, pois substitui totalmente a assinatura do token.

14. Em relação a SAML e OpenID Connect, assinale a alternativa correta no contexto de SSO corporativo.

- (A) SAML e OpenID Connect são nomes diferentes para o mesmo formato de token JSON.
- (B) SAML é uma ferramenta de backup de sessão, enquanto OpenID Connect é um protocolo de banco de dados.
- (C) OpenID Connect não pode ser usado com Keycloak, pois Keycloak suporta apenas SAML.
- (D) SAML elimina a necessidade de relação de confiança entre provedor e aplicação.
- (E) SAML é tradicionalmente baseado em assertions XML; OpenID Connect adiciona autenticação sobre OAuth2 e usa conceitos como ID Token e claims.

15. Um servidor público acessa uma aplicação integrada ao Keycloak. Após informar usuário e senha, é solicitado um segundo fator em aplicativo autenticador. O objetivo principal desse segundo fator é:

- (A) substituir autorização por autenticação, liberando todos os perfis do sistema.
- (B) elevar a garantia da autenticação, exigindo mais de uma categoria de evidência para comprovar a identidade.
- (C) criptografar o banco de dados da aplicação automaticamente.
- (D) transformar o refresh token em access token permanente.
- (E) dispensar TLS, pois MFA protege o tráfego em rede.

16. Uma API Quarkus protegida por Keycloak recebe access tokens enviados por clientes. Considerando uma configuração segura, assinale a opção que representa uma validação adequada pela API.

- (A) Aceitar o token se o usuário existir na tela de administração, sem conferir assinatura.
- (B) Aceitar qualquer token com três partes separadas por ponto, pois isso caracteriza JWT válido.
- (C) Validar assinatura com a chave pública esperada, issuer, audience, expiração e roles/scopes necessários à operação.

- (D) Validar apenas o campo nome, pois claims de identificação substituem assinatura.
- (E) Exigir que o refresh token seja enviado em todas as requisições de API.

17. Assinale a opção que apresenta uma configuração que tende a aumentar o risco em ambiente SSO com Keycloak.

- (A) Access tokens com vida curta e refresh tokens protegidos por política de rotação.
- (B) MFA em operações sensíveis e políticas de senha centralizadas.
- (C) Validação de issuer e audience nas APIs protegidas.
- (D) Redirect URIs configuradas de forma ampla, permitindo retorno para origens não controladas.
- (E) Uso de TLS nas comunicações entre navegador, IdP e aplicações.

18. Analise as afirmações: I. Em SSO, a aplicação cliente deve necessariamente conhecer e armazenar a senha do usuário para reaproveitá-la nos sistemas integrados. II. Em Keycloak, token mappers podem influenciar quais atributos ou roles aparecem nos tokens. III. Em autenticação federada, o mapeamento inadequado de atributos recebidos de IdP externo pode causar concessão indevida de perfil. Está correto o que se afirma em:

- (A) I, apenas.
- (B) I e II, apenas.
- (C) I e III, apenas.
- (D) II e III, apenas.
- (E) I, II e III.

19. No desenho de uma arquitetura para múltiplas aplicações internas, decidiu-se que o Keycloak será o ponto central de autenticação, enquanto cada aplicação continuará responsável por regras específicas de negócio. Essa decisão está alinhada com a ideia de que:

- (A) autenticação centralizada pode conviver com autorização específica por aplicação ou API.
- (B) SSO impede qualquer regra de autorização local.
- (C) o ID token deve ser usado como refresh token em todas as APIs.
- (D) as aplicações devem receber a senha do usuário para validar sessão.
- (E) roles devem ser substituídas por cookies sem expiração.

20. Uma aplicação recebe um ID token após autenticação do usuário. O desenvolvedor pretende usá-lo como bearer token para autorizar acesso a uma API de dados sigilosos. A melhor avaliação é:

- (A) correta, pois ID token e access token têm sempre o mesmo público e a mesma finalidade.
- (B) incorreta, pois o ID token é destinado à aplicação cliente para informações de autenticação/identidade, enquanto a API deve receber token apropriado para acesso ao recurso.
- (C) correta, desde que o usuário tenha sessão SSO ativa no navegador.
- (D) incorreta apenas se o token estiver assinado; tokens não assinados podem ser usados em APIs.
- (E) correta se o payload estiver em Base64URL.

21. Em prova, a FGV apresenta o seguinte cenário: 'Usuários autenticam em um provedor central; aplicações não veem as credenciais; tokens ou assertions assinadas carregam informações de identidade e permissões; APIs validam artefatos antes de liberar recursos'. A descrição combina principalmente os conceitos de:

- (A) particionamento de banco, replicação síncrona e normalização.
- (B) balanceamento de carga, cache de página e compressão de logs.
- (C) SSO, provedor de identidade, federação/authenticação centralizada e validação de tokens/assertions.
- (D) criptografia de disco, backup incremental e restore point.
- (E) compilação ahead-of-time, árvore B e trigger de auditoria.

22. Gabarito resumido

Gabarito
1-C, 2-E, 3-B, 4-D, 5-A
6-C, 7-B, 8-E, 9-D, 10-A
11-C, 12-A, 13-D, 14-E, 15-B
16-C, 17-D, 18-D, 19-A, 20-B

23. Comentários completos

Questão 1 - Gabarito: C

A correta é C: SSO centraliza ou federa a autenticação para múltiplas aplicações confiantes. A erra porque rede corporativa não dispensa autenticação. B confunde autenticação com autorização: login não libera tudo. D transforma SSO em criptografia de tráfego; quem protege canal é TLS. E inverte a confiança: a aplicação confia no IdP, não vira autoridade emissora para as demais.

Análise alternativa por alternativa:

- (A) A erra porque localização na rede não autentica identidade.
- (B) B erra porque autorização continua necessária.
- (C) C é a correta: SSO centraliza/federa login para apps confiantes.
- (D) D confunde SSO com criptografia de canal.
- (E) E inverte o papel: app confia no IdP, não vira IdP automaticamente.

Erro provável: confusão entre autenticação e autorização.

Questão 2 - Gabarito: E

E é correta: Keycloak atua como servidor de identidade e acesso e suporta padrões como OIDC e SAML. A é errada: não é biblioteca de front-end nem deve armazenar senha no navegador. B é perigosa: o backend continua validando tokens e autorização. C erra porque Keycloak não é protocolo concorrente; é uma solução que implementa protocolos. D contraria a arquitetura típica: Keycloak é servidor separado.

Análise alternativa por alternativa:

- (A) A reduz Keycloak a biblioteca de front-end, o que é falso.
- (B) B ignora validação no backend.
- (C) C chama Keycloak de protocolo, erro clássico.
- (D) D nega o modelo de servidor separado.
- (E) E é correta: Keycloak atua como servidor IAM/IdP e suporta padrões.

Erro provável: confusão entre componentes do Keycloak e papéis da arquitetura.

Questão 3 - Gabarito: B

B é correta: o Keycloak pode atuar como broker entre aplicação e outro IdP. A é tema de criptografia, não federação. C sugere cache de credenciais, justamente o que se evita. D fala de backup de sessão, sem relação. E cita autorização discricionária, mas o caso é delegação de autenticação.

Análise alternativa por alternativa:

- (A) A é tema de cifra, não de federação.
- (B) B é correta: Keycloak intermedeia identidade com IdP externo.
- (C) C sugere cache de credenciais, justamente o que se evita.
- (D) D trata sessão como backup, sem relação.
- (E) E fala de autorização discricionária, outro assunto.

Erro provável: confusão entre componentes do Keycloak e papéis da arquitetura.

Questão 4 - Gabarito: D

II e III estão corretas. I é a armadilha: SSO não elimina autorização. II é compatível com o papel do Keycloak. III descreve federação com confiança e validação. A, B e C incluem I, que está errada. E também inclui I.

Análise alternativa por alternativa:

- (A) A mantém I, que está errada.
- (B) B mantém I, que está errada.
- (C) C mantém I, que está errada.
- (D) D é correta: II e III descrevem Keycloak e federação.
- (E) E inclui I, que confunde SSO com autorização total.

Erro provável: confusão entre autenticação e autorização.

Questão 5 - Gabarito: A

A distingue corretamente user federation e identity brokering. B mistura assinatura e criptografia. C inventa restrição por arquitetura. D trata realms e clients como substituíveis, o que é falso. E confunde esses conceitos com logout e refresh token.

Análise alternativa por alternativa:

- (A) A é correta: distingue fonte externa de usuários de broker de IdP.
- (B) B mistura assinatura e criptografia.
- (C) C inventa restrição por arquitetura.
- (D) D confunde componentes do Keycloak.
- (E) E mistura conceitos com logout/refresh token.

Erro provável: confusão entre componentes do Keycloak e papéis da arquitetura.

Questão 6 - Gabarito: C

C é correta: assinatura garante integridade/autenticidade do emissor, não confidencialidade. A é falsa: assinatura não cifra conteúdo. B é absurda no sentido técnico: HTTP sem TLS reduz segurança. D inventa diferença entre OIDC e OAuth2. E aud não torna payload secreto.

Análise alternativa por alternativa:

- (A) A erra: assinatura não cifra conteúdo.
- (B) B erra e ainda remove TLS.
- (C) C é correta: assinatura garante integridade/autenticidade, não sigilo.
- (D) D cria distinção inexistente.
- (E) E aud não torna payload secreto.

Erro provável: lacuna técnica em validação de tokens ou segurança de fluxo.

Questão 7 - Gabarito: B

A sequência é 3 - 2 - 1. Refresh token renova tokens; ID token comunica autenticação/identidade ao client; access token é apresentado à API. As demais alternativas trocam finalidades, pegadinha clássica entre ID token e access token.

Análise alternativa por alternativa:

- (A) A coloca access token como renovação, errado.
- (B) B é correta: refresh, ID, access.
- (C) C troca ID e access.
- (D) D coloca ID no lugar de refresh.
- (E) E troca finalidade do ID token.

Erro provável: memorização superficial de siglas sem distinguir finalidade.

Questão 8 - Gabarito: E

E é correta: token precisa ser validado de verdade, não apenas lido. A erra porque APIs podem validar tokens sem sessão local. B é falsa: expiração é medida de segurança. C ataca SSO indevidamente. D é falsa: claims são centrais, mas devem ser validadas.

Análise alternativa por alternativa:

- (A) A erra porque API pode ser stateless e validar token.
- (B) B erra: expiração é controle.
- (C) C culpa indevidamente o SSO.
- (D) D nega utilidade de claims.
- (E) E é correta: validação precisa ser completa.

Erro provável: lacuna técnica em validação de tokens ou segurança de fluxo.

Questão 9 - Gabarito: D

D separa sessão e token. A é falsa e reduz ambos a senha. B confunde autenticação com autorização. C é falsa: logout local não encerra automaticamente sessão global. E erra porque SSO normalmente envolve sessão no provedor.

Análise alternativa por alternativa:

- (A) A reduz sessão/token a senha, falso.
- (B) B dispensa autorização indevidamente.
- (C) C confunde logout local e global.
- (D) D é correta: sessão é estado, token é artefato.
- (E) E nega sessões no SSO, falso.

Erro provável: confusão entre autenticação e autorização.

Questão 10 - Gabarito: A

A é correta: autorização precisa estar no backend/API. B é falsa: Angular pode integrar com Keycloak. C erra: roles são de IAM/autorização, não exclusivas de banco. D inventa exigência de cookie. E é o oposto do correto: SSO não impede controle de acesso.

Análise alternativa por alternativa:

- (A) A é correta: backend deve validar autorização.
- (B) B é falsa: Angular pode usar Keycloak.
- (C) C restringe roles a banco, errado.
- (D) D inventa exigência de cookies.
- (E) E erra: SSO convive com controle de acesso.

Erro provável: confusão entre autenticação e autorização.

Questão 11 - Gabarito: C

I e II estão corretas; III erra porque role não autentica, apenas apoia autorização. A e B são incompletas. D inclui III, errada. E também inclui III.

Análise alternativa por alternativa:

- (A) A ignora client.
- (B) B ignora realm.
- (C) C é correta: realm e client estão certos; role não autentica.
- (D) D inclui III, falsa.
- (E) E inclui III, falsa.

Erro provável: confusão entre componentes do Keycloak e papéis da arquitetura.

Questão 12 - Gabarito: A

A explica o cenário: o logout local não matou a sessão central. B é tecnicamente equivocada. C é falsa, Keycloak suporta mecanismos de logout. D erra porque JWT pode expirar. E foge do caso.

Análise alternativa por alternativa:

- (A) A é correta: a sessão central permaneceu.
- (B) B confunde token e senha.
- (C) C é falso: Keycloak suporta logout.
- (D) D afirma que JWT nunca expira, falso.
- (E) E foge do desenho com IdP.

Erro provável: confusão entre componentes do Keycloak e papéis da arquitetura.

Questão 13 - Gabarito: D

D é correta: audience indica o destinatário do token. A, B e C atribuem funções erradas a iat, sub e exp. E é falso: nonce ajuda contra certos ataques em fluxos OIDC, mas não substitui assinatura nem define público.

Análise alternativa por alternativa:

- (A) A dá função errada ao iat.
- (B) B dá função errada ao sub.
- (C) C dá função errada ao exp.
- (D) D é correta: audience delimita o destinatário.
- (E) E superestima nonce.

Erro provável: lacuna técnica em validação de tokens ou segurança de fluxo.

Questão 14 - Gabarito: E

E é correta: SAML usa assertions XML; OIDC é camada de identidade sobre OAuth2, com ID Token e claims. A os iguala indevidamente. B foge do domínio. C é falsa: Keycloak suporta OIDC. D é falsa: SAML exige relação de confiança.

Análise alternativa por alternativa:

- (A) A iguala padrões diferentes.
- (B) B inventa natureza dos protocolos.
- (C) C nega suporte do Keycloak a OIDC.

(D) D elimina a confiança, que é essencial.

(E) E é correta: SAML XML; OIDC autenticação sobre OAuth2.

Erro provável: memorização superficial de siglas sem distinguir finalidade.

Questão 15 - Gabarito: B

B é correta: MFA eleva garantia de autenticação combinando fatores. A confunde autenticação e autorização. C atribui efeito inexistente em banco. D é falsa: refresh token não vira access token permanente. E é perigosa: MFA não substitui TLS.

Análise alternativa por alternativa:

(A) A confunde autorização com autenticação.

(B) B é correta: MFA aumenta garantia de autenticação.

(C) C atribui efeito em banco inexistente.

(D) D cria token permanente, inseguro.

(E) E substitui TLS por MFA, erro grave.

Erro provável: memorização superficial de siglas sem distinguir finalidade.

Questão 16 - Gabarito: C

C lista validações esperadas: assinatura, issuer, audience, expiração e autorização. A e B aceitam token sem validação suficiente. D confunde claim com prova criptográfica. E é errado: API deve receber access token, não refresh token.

Análise alternativa por alternativa:

(A) A aceita usuário sem prova criptográfica.

(B) B confunde formato com validade.

(C) C é correta: validações essenciais.

(D) D confunde claim com assinatura.

(E) E expõe refresh token indevidamente.

Erro provável: lacuna técnica em validação de tokens ou segurança de fluxo.

Questão 17 - Gabarito: D

D aumenta risco porque redirect URI ampla pode permitir desvios de fluxo e entrega indevida de artefatos. A, B, C e E são práticas de mitigação ou controle, não de aumento de risco.

Análise alternativa por alternativa:

(A) A reduz risco.

(B) B reduz risco.

(C) C reduz risco.

(D) D é correta: redirect URI ampla aumenta risco.

(E) E reduz risco.

Erro provável: lacuna técnica em validação de tokens ou segurança de fluxo.

Questão 18 - Gabarito: D

II e III estão corretas. I erra porque aplicações não devem conhecer/reaproveitar senha do usuário. Mappers realmente influenciam claims/roles; mapeamento ruim em federação pode gerar privilégio indevido. As alternativas com I estão erradas.

Análise alternativa por alternativa:

(A) A inclui I, falsa.

(B) B inclui I, falsa.

(C) C inclui I, falsa.

(D) D é correta: II e III estão certas.

(E) E inclui I, falsa.

Erro provável: confusão entre componentes do Keycloak e papéis da arquitetura.

Questão 19 - Gabarito: A

A é correta: autenticação centralizada pode coexistir com autorização por sistema/API. B é falsa: SSO não impede autorização local. C troca ID token por refresh token. D contraria o modelo de delegação de autenticação. E cria cookie sem expiração, prática insegura.

Análise alternativa por alternativa:

- (A) A é correta: autenticação central pode coexistir com autorização local.
- (B) B nega autorização local.
- (C) C troca tokens.
- (D) D exige senha na aplicação, inadequado.
- (E) E propõe cookie sem expiração.

Erro provável: confusão entre autenticação e autorização.

Questão 20 - Gabarito: B

B é correta: ID token é para o client obter prova de autenticação e claims; API deve receber token apropriado de acesso. A é a confusão clássica. C não muda finalidade do token. D é absurda: token não assinado é menos confiável. E Base64URL é codificação, não autorização.

Análise alternativa por alternativa:

- (A) A iguala ID token e access token, erro clássico.
- (B) B é correta: API deve receber token apropriado de acesso.
- (C) C sessão SSO não muda finalidade do token.
- (D) D sugere token não assinado, inseguro.
- (E) E confunde codificação com autorização.

Erro provável: confusão entre autenticação e autorização.

24. Checklist final do que memorizar

- SSO = autenticação centralizada/federada, não ausência de autenticação.
- Autenticação prova identidade; autorização concede acesso; identificação apenas declara quem o usuário diz ser.
- Keycloak = servidor IAM/IdP/broker, não protocolo.
- Realm isola domínio de segurança; client representa aplicação/serviço; role apoia autorização.
- User federation integra diretórios externos; identity brokering delega autenticação a outro IdP.
- Access token vai para API/recurso; ID token vai para client; refresh token renova tokens.
- JWT assinado não é confidencial por padrão.
- API deve validar assinatura, issuer, audience, expiração e permissões.
- Logout local não implica automaticamente logout global no IdP.
- Front-end não é ponto suficiente de autorização; backend precisa validar.

25. Referências consultadas

- TJSC - Edital n. 10/2026 retificado em 10/04/2026, especialmente itens de Segurança da Informação e Governança de TI.
- Manual Mestre de Calibração FGV para Aulas e Questões - Projeto TJSC 2026.
- FGV Conhecimento - caderno público TJRR Analista Judiciário - Cibersegurança, com item envolvendo SAML, SSO, Keycloak, OpenID Connect e OAuth 2.0.
- Keycloak Documentation - Server Administration Guide e página oficial do projeto Keycloak.
- IETF - About RFCs e RFC 6749 - The OAuth 2.0 Authorization Framework, como contexto para a próxima aula.
- OpenID Foundation - OpenID Connect Core 1.0, como contexto para distinção entre ID token e access token.